Université Abdelmalek Essaâdi
Faculté des Sciences et Techniques de Tanger
Département Génie Informatique
MST : Systèmes Intelligents et Cybersécurité

Module : *Développement Sécurisé des Applications Mobiles*

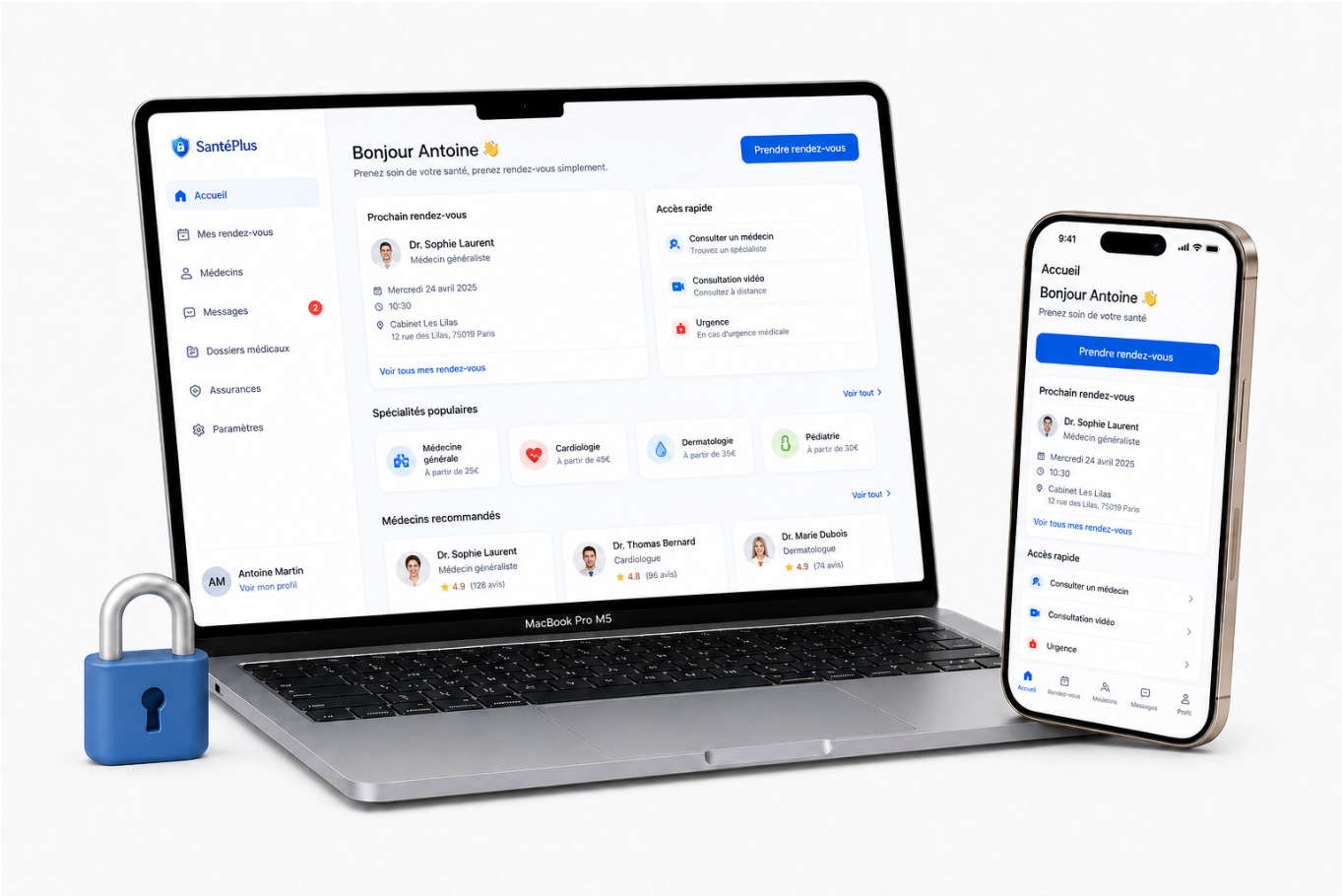
RAPPORT D'AUDIT DE SÉCURITÉ & REMÉDIATION

Sécurisation de l'Application SantéPlus (React Native)

Conformité OWASP Mobile Top 10 (2024), MASVS & Loi 09-08

Réalisé par :
EL BOURMAKI SALIM

Encadré par :
Pr. ZILI HASSAN



Année Universitaire : 2026 / 2027

Table des matières

1 Résumé Exécutif **1**

2 Méthodologie d'Audit **1**
2.1 Périmètre 1
2.2 Approche 1
2.3 Outils utilisés 1
2.4 Grille de criticité 1

3 Concepts Fondamentaux **1**
3.1 Taxonomie OWASP Mobile Top 10 (2024) 1
3.2 AsyncStorage vs Keychain iOS / Android Keystore 2
3.3 Fichiers `.env` et Bundling : l'Illusion du Secret 2
3.4 Certificate Pinning : Bénéfice et Risque Opérationnel 2

4 Audit de Code **2**

5 Remédiation : Architecture *santeplus-secure* **3**
5.1 Stockage Sécurisé (`secureStorage.ts`) 3
5.2 Authentification et Client API (`auth.ts`, `api.ts`) 3
5.3 Contrôle d'Accès et Correction de `isAdmin()` 4
5.4 Deep Linking Résilient (`linking.ts`) 4
5.5 WebView Sécurisée (`ArticleScreen.tsx`) 5
5.6 Authentification Biométrique des Ordonnances (`OrdonnancesScreen.tsx`) 5
5.7 Bonus : Proxy Sécurisé pour l'API Google Maps (+1 pt) 5

6 Analyses Critiques **6**
6.1 Hermes Bytecode : Optimisation ou Protection ? 6
6.2 Détection Root / Jailbreak : Portée Réelle 6
6.3 Mises à Jour OTA (EAS Update) : Risques et Durcissement 6

7 Plan de Vérification **6**

8 Glossaire **7**

9 Conclusion et Perspectives **7**

1. Résumé Exécutif

Ce rapport présente l'audit de sécurité complet de l'application mobile **SantéPlus**, développée en React Native / Expo, ainsi que le plan de durcissement ayant conduit à la version `santeplus-secure`.

L'audit a identifié **14 vulnérabilités** réparties sur six catégories du référentiel **OWASP Mobile Top 10 (2024)**, dont **11 de criticité élevée**. Les failles les plus critiques concernent :

- Le stockage en clair de jetons JWT et de mots de passe dans `AsyncStorage` (M9)
- L'utilisation de HTTP non chiffré pour les échanges de données de santé (M5)
- L'exposition de la clé d'API Google Maps en dur dans le bundle JavaScript (M1)
- L'autorisation administrative vérifiée exclusivement côté client (M3)

La remédiation repose sur quatre piliers : chiffrement du stockage via `expo-secure-store`, généralisation de TLS avec en-têtes Bearer (RFC 6750), délégation de l'autorisation au serveur via des jetons JWT signés, et suppression de tout secret du binaire client via un proxy backend. L'architecture résultante est conforme à l'**OWASP MASVS v2.0** et à l'**article 23 de la Loi 09-08** relative à la protection des données personnelles au Maroc.

2. Méthodologie d'Audit

2.1. Périmètre

L'audit couvre l'intégralité du code source de l'application React Native (composants, services, configuration), le fichier de configuration Expo `app.json`, ainsi que les flux réseau entre le client mobile et l'API backend.

2.2. Approche

L'audit combine deux méthodes complémentaires :

- **Analyse statique (SAST) :** revue manuelle du code source ligne par ligne, complétée par un scan automatisé **MobSF** (Mobile Security Framework) sur l'APK de production et le bundle iOS.
- **Analyse dynamique (DAST) :** interception des flux réseau via **mitmproxy** sur un terminal Android rooté (Pixel 6, Android 14), instrumentation runtime avec **Frida** pour vérifier la lisibilité du stockage local, et extraction du bundle JavaScript via `apktool`.

2.3. Outils utilisés

Outil	Usage
MobSF v4.0	Scan statique automatisé APK / IPA
apktool v2.9	Décompilation de l'APK et extraction du bundle JS
mitmproxy v10	Interception et inspection des flux HTTPS
Frida v16	Instrumentation dynamique (hook de fonctions JS et natives)
Wireshark v4.2	Capture passive sur réseau Wi-Fi hospitalier simulé
adb v34	Accès au système de fichiers et aux logs du terminal
jadx v1.5	Décompilation du code natif Android

2.4. Grille de criticité

La criticité de chaque vulnérabilité est évaluée selon une matrice **Risque = Probabilité** $\times$ **Impact** inspirée du modèle CVSS v3.1 :

Niveau	Critères	Score
Élevée	Exploitation triviale, données de santé exposées, contournement d'authentification	7.0 – 10.0
Moyenne	Exploitation nécessitant un accès local ou une configuration particulière	4.0 – 6.9
Faible	Impact limité, exploitation complexe ou théorique	0.1 – 3.9

3. Concepts Fondamentaux

3.1. Taxonomie OWASP Mobile Top 10 (2024)

La taxonomie 2024 recense dix familles de risques spécifiques aux plateformes mobiles :

- **M1 : Improper Credential Usage :** secrets et clés d'API codés en dur, tokens transmis sans protection
- **M2 : Inadequate Supply Chain Security :** dépendances tierces et pipeline CI/CD non maîtrisés

- **M3 : Insecure Authentication / Authorization :** contrôle d'accès défaillant, logique d'élévation côté client
- **M4 : Insufficient Input/Output Validation :** injection, traversée de chemin, désérialisation non contrôlée
- **M5 : Insecure Communication :** transport HTTP sans TLS, dégradation MITM du handshake
- **M6 : Inadequate Privacy Controls :** collecte et conservation excessives de données personnelles
- **M7 : Insufficient Binary Protections :** résistance nulle au reverse engineering et à l'instrumentation dynamique
- **M8 : Security Misconfiguration :** flags de débogage ou permissions permissives maintenus en production
- **M9 : Insecure Data Storage :** données sensibles persistées en clair sur le terminal
- **M10 : Insufficient Cryptography :** algorithmes faibles, clés en dur, implémentations maison défectueuses

Trois catégories directement impliquées dans SantéPlus. **M1** désigne l'exposition de secrets directement dans le code source ou leur acheminement via des canaux non chiffrés. La clé Google Maps présente en clair dans `config.js` est extractible par simple décompilation APK, engageant à la fois la facturation de l'organisation et la confidentialité des adresses de patients. **M5** cible les applications véhiculant des données de santé sans garanties d'intégrité TLS : un attaquant passif sur Wi-Fi hospitalier capture identifiants et tokens en quelques secondes avec Wireshark. **M9** vise le stockage de données confidentielles dans des conteneurs non chiffrés. SantéPlus conservant mots de passe et jetons dans `AsyncStorage`, tout malware s'exécutant avec des privilèges élevés accède au dossier médical sans obstacle cryptographique.

3.2. AsyncStorage vs Keychain iOS / Android Keystore

`AsyncStorage` est un magasin clé-valeur non chiffré : les données persistent sous forme de texte brut dans une base SQLite (Android) ou des fichiers sérialisés (iOS), accessibles à quiconque dispose d'un accès ADB ou d'une élévation de privilèges (*root / jailbreak*).

Le **Keychain iOS** et l'**Android Keystore** reposent sur des puces cryptographiques matériellement isolées (Secure Enclave, TEE, StrongBox) offrant chiffrement AES-256-GCM et attestation de clé. La politique `WHEN_UNLOCKED_THIS_DEVICE_ONLY` maintient les clés inaccessibles lorsque l'appareil est verrouillé et les exclut automatiquement des sauvegardes iCloud / Google Drive. Stocker des jetons ou des données médicales dans `AsyncStorage` constitue une violation formelle de l'article 23 de la Loi 09-08.

3.3. Fichiers `.env` et Bundling : l'Illusion du Secret

Une variable issue d'un `.env` n'est jamais secrète dans une application mobile. Lors de la compilation (Metro Bundler), chaque occurrence de `process.env.VAR` est substituée en clair dans le bundle JavaScript final (`index.android.bundle`), empaqueté sans chiffrement dans l'APK. La commande `strings index.android.bundle | grep AIza` extrait la clé Google Maps en quelques secondes. Un secret critique doit résider exclusivement sur un serveur d'arrière-plan exposant un *Backend Proxy* ; il ne doit jamais transiter dans un artefact déployé sur le terminal.

3.4. Certificate Pinning : Bénéfice et Risque Opérationnel

Le **Certificate Pinning** (épinglage SPKI) force l'application à vérifier l'empreinte cryptographique du certificat serveur contre une valeur de confiance intégrée au binaire, neutralisant les attaques Man-in-the-Middle reposant sur l'installation d'une autorité de certification frauduleuse (proxy d'inspection d'entreprise, CA compromise).

Le risque opérationnel est proportionnel à la rigueur du mécanisme : un renouvellement de certificat non synchronisé avec une mise à jour applicative entraîne un refus catégorique de toute connexion HTTPS, soit un déni de service complet pour les patients. La pratique recommandée consiste à épingler la clé publique SPKI d'un certificat de secours (*backup pin*) et à gérer le renouvellement des empreintes via des mises à jour OTA signées cryptographiquement.

4. Audit de Code

L'examen des cinq extraits fournis révèle **14 vulnérabilités distinctes** sur six catégories OWASP Mobile Top 10 (2024).

#	Fichier / Ligne	Vulnérabilité	OWASP 2024	Scénario d'exploitation	Criticité
1	`config.js` L1	Endpoint API en HTTP	M5	Capture passive (Wireshark) sur Wi-Fi hospitalier ; tokens médicaux et corps de réponse interceptés en clair.	Élevée
2	`config.js` L2	Clé Google Maps codée en dur	M1	Extraction via `apktool` en quelques secondes ; réutilisation illimitée aux frais de l'organisation.	Élevée
3	`config.js` L3	`DEBUG=true` en production	M8	Exfiltration complète des flux et tokens de session via `adb logcat -s ReactNativeJS`.	Moyenne
4	`auth.js` L5	Identifiants dans la query string (GET)	M5	Mots de passe archivés en clair dans les journaux Nginx, les proxys d'entreprise et l'historique navigateur.	Élevée
5	`auth.js` L7	Journalisation de tokens JWT	M9	Fuite vers les plateformes d'observabilité (Sentry, Crashlytics) et la console système locale.	Moyenne
6	`auth.js` L8	Token JWT dans `AsyncStorage`	M9	Vol du jeton non chiffré depuis le fichier SQLite local via `adb backup` sur terminal rooté.	Élevée
7	`auth.js` L9	Mot de passe stocké en clair	M9	Accès physique au terminal ; extraction du mot de passe maître du patient et compromission de ses autres comptes.	Élevée
8	`auth.js` L13-15	Autorisation `isAdmin()` côté client	M3	Élévation de privilèges par modification de la clé `role` via Frida ou SQLite ; aucune vérification serveur.	Élevée
9	`linking.js` L2	Paramètre `id` non validé	M4	Lien SMS forgé (`santeplus://rdv?id=../../admin`) ; traversée de chemin atteignant l'API backend.	Élevée
10	`linking.js` L4	Token d'accès dans l'URL du deep link	M1	Fuite du jeton dans l'en-tête `Referer` et les journaux des mandataires réseau intermédiaires.	Élevée
11	`linking.js` L1-5	Schéma custom sans garde d'authentification	M3	Application tierce enregistrant `santeplus://` ; interception de liens RDV par Link Hijacking.	Élevée
12	`ArticleScreen` L4	`originWhitelist=["*"]`	M8	Redirection vers une page de phishing hébergée au sein de la WebView légitime de l'application.	Élevée
13	`ArticleScreen` L5	Exécution arbitraire via `eval()`	M4	Injection JavaScript (RCE) via `postMessage` ; exfiltration des données applicatives vers un serveur C2.	Élevée
14	`AndroidManifest` L2-4	Flags de manifest permissifs	M5, M8, M9	`usesCleartextTraffic` réactive HTTP, `allowBackup` permet l'extraction USB, `debuggable` autorise l'attachement d'un débogueur tiers.	Élevée

5. Remédiation : Architecture *santeplus-secure*

5.1. Stockage Sécurisé (`secureStorage.ts`)

`AsyncStorage` est définitivement remplacé par `expo-secure-store`, configuré avec la politique `WHEN_UNLOCKED_THIS_DEV` les clés cryptographiques sont inaccessibles lorsque l'appareil est verrouillé et ne peuvent jamais être exportées. Le mot de passe en clair utilisé pour la reconnexion automatique est supprimé et remplacé par un cycle **Access Token (15 min) / Refresh Token (7 jours)**, conforme à la pratique OAuth 2.0.

Avant (vulnérable)

```
// auth.js -- VULNERABLE (M9)
await AsyncStorage.setItem("token", data.token);
await AsyncStorage.setItem("password", password); // Stockage mot de passe en clair !
```

Listing 1 – Preuve d'implémentation : src/services/secureStorage.ts

Après (sécurisé)

```
import * as SecureStore from "expo-secure-store";

const KEYS = {
  ACCESS_TOKEN: "santeplus_access_token",
  REFRESH_TOKEN: "santeplus_refresh_token",
} as const;

const SECURE_OPTIONS: SecureStore.SecureStoreOptions = {
  keychainAccessible: SecureStore.WHEN_UNLOCKED_THIS_DEVICE_ONLY,
};

export async function saveTokens(accessToken: string, refreshToken: string): Promise<void> {
  await SecureStore.setItemAsync(KEYS.ACCESS_TOKEN, accessToken, SECURE_OPTIONS);
  await SecureStore.setItemAsync(KEYS.REFRESH_TOKEN, refreshToken, SECURE_OPTIONS);
}

export async function getAccessToken(): Promise<string | null> {
  return SecureStore.getItemAsync(KEYS.ACCESS_TOKEN, SECURE_OPTIONS);
}
```

5.2. Authentification et Client API (`auth.ts`, `api.ts`)

Les identifiants transitent désormais exclusivement via un corps JSON sérialisé sur **POST HTTPS**, satisfaisant MASVS-NETWORK-1. Les jetons sont transmis via l'en-tête standard `Authorization: Bearer` (RFC 6750) ; un intercepteur gère leur régénération transparente à l'expiration (HTTP 401). Les logs sont épurés de toute donnée sensible grâce à une bascule conditionnelle sur le flag natif `__DEV__`.

Avant (vulnérable)

```
// auth.js -- VULNERABLE (M5, M9)
```

```
const res = await fetch(`${API_URL}/login?email=${email}&password=${password}`);
if (DEBUG) console.log("Login OK", data); // Fuite du token et mot de passe dans logcat
```

Listing 2 – Preuve d'implémentation : src/services/auth.ts

Après (sécurisé)

```
export async function login(email: string, password: string): Promise<LoginResponse["user"]> {
  const response = await fetch(`${API_BASE_URL}/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });
  if (!response.ok) throw new Error("Echec de la connexion");
  const data: LoginResponse = await response.json();
  await saveTokens(data.accessToken, data.refreshToken);
  if (__DEV__) console.log(`[Auth] Connexion reussie : ${data.user.email}`);
  return data.user;
}
```

Listing 3 – Preuve d'implémentation : src/services/api.ts

```
export async function apiRequest<T>(endpoint: string, options: RequestInit = {}): Promise<T> {
  const accessToken = await getAccessToken();
  const headers: HeadersInit = {
    "Content-Type": "application/json",
    ...(options.headers || {}),
    ...(accessToken ? { Authorization: `Bearer ${accessToken}` } : {}),
  };
  const response = await fetch(`${API_BASE_URL}${endpoint}`, { ...options, headers });
  if (response.status === 401 && !isRefreshing) {
    const refreshed = await attemptTokenRefresh();
    if (refreshed) return apiRequest<T>(endpoint, options);
    await clearTokens();
    throw new Error("SESSION_EXPIRED");
  }
  return response.json();
}
```

5.3. Contrôle d'Accès et Correction de `isAdmin()`

La vulnérabilité originelle reposait sur la lecture de `AsyncStorage.getItem("role")` côté client pour accorder les droits administrateur. Sur un terminal rooté, une simple instruction SQLite suffit à s'octroyer ce rôle :

```
UPDATE catalystLocalStorage SET value = '"admin"' WHERE key = '"role"';
```

La remédiation applique le principe fondamental *Never Trust the Client* : le rôle est désormais encapsulé dans les *claims* d'un JWT signé côté serveur. Un middleware Express (`authenticateToken`) décode et valide le jeton avant chaque accès à un endpoint protégé (`GET /v1/rdv/:id`), indépendamment de toute donnée transmise par le client.

Listing 4 – Preuve d'implémentation : server/server.js

Après (sécurisé)

```
function authenticateToken(req, res, next) {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.startsWith("Bearer ") ? authHeader.split(" ")[1] : null;
  if (!token) return res.status(401).json({ error: "Token manquant." });
  try {
    req.user = jwt.verify(token, JWT_SECRET); // Validation cryptographique HMAC-SHA256
    next();
  } catch (err) {
    return res.status(403).json({ error: "Token invalide ou expire." });
  }
}

app.get("/v1/rdv/:id", authenticateToken, (req, res) => {
  const rdv = RENDEZ_VOUS.find((r) => r.id === req.params.id);
  if (!rdv) return res.status(404).json({ error: "Rendez-vous introuvable." });
  // RBAC Objet : Un patient ne peut consulter que ses propres dossiers
  if (req.user.role !== "admin" && rdv.userId !== req.user.userId) {
    return res.status(403).json({ error: "Acces interdit a ce rendez-vous." });
  }
  res.json(rdv);
});
```

5.4. Deep Linking Résilient (`linking.ts`)

La remédiation du routage par deep link repose sur trois mesures complémentaires :

- **Validation UUID v4 :** l'identifiant de rendez-vous est validé par la regex suivante avant tout passage à l'API backend, éliminant les tentatives de path traversal et d'injection :

```
^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$
```

- **Garde de session :** la présence d'un token valide dans le Keystore est vérifiée avant toute redirection d'écran.
- **App Links et Universal Links :** les schémas custom (`santeplus://`) sont abandonnés au profit des **Android App Links** et **iOS Universal Links**, dont l'authenticité est garantie par les fichiers `assetlinks.json` et `apple-app-site-association` hébergés sur le domaine maîtrisé de l'organisation, rendant le Link Hijacking structurellement impossible.

Listing 5 – Preuve d'implémentation : src/navigation/linking.ts

```tsx
export async function handleDeepLink(id: string, navigation: any): Promise<void> {
  if (!UUID_V4_REGEX.test(id)) return; // Rejet immediat des formats invalides
  const token = await getAccessToken();
  if (!token) {
    navigation.navigate("Login", { redirect: `rdv/${id}` });
    return;
  }
  const rdv = await apiRequest(`/rdv/${id}`);
  navigation.navigate("Rdv", { rdv });
}
```

5.5. WebView Sécurisée (`ArticleScreen.tsx`)

L'origine autorisée est restreinte à `https://santeplus.ma` via la propriété `originWhitelist`. L'appel à `eval()` est remplacé par un pont d'échange `onMessage` : chaque message entrant est parsé par `JSON.parse()` et filtré contre une liste blanche d'actions autorisées (`ARTICLE_LOADED`, `SHARE_ARTICLE`, `ARTICLE_ERROR`), rendant toute injection JavaScript extérieure sans effet.

Listing 6 – Preuve d'implémentation : src/screens/ArticleScreen.tsx

```tsx
const ALLOWED_ACTIONS = ["ARTICLE_LOADED", "SHARE_ARTICLE", "ARTICLE_ERROR"] as const;

const handleMessage = useCallback((event: { nativeEvent: { data: string } }) => {
  let message: { type?: string; payload?: unknown };
  try {
    message = JSON.parse(event.nativeEvent.data);
  } catch { return; } // Blocage du JavaScript brut injecte
  if (!message.type || !(ALLOWED_ACTIONS as readonly string[]).includes(message.type)) {
    return; // Rejet silencieux des actions non declarees
  }
}, []);
```

5.6. Authentification Biométrique des Ordonnances (`OrdonnancesScreen.tsx`)

Conformément à l'article 23 de la Loi 09-08, l'accès aux ordonnances est conditionné à une authentification forte via `expo-local-authentication` selon la séquence suivante : vérification de la présence matérielle du capteur (`hasHardwareAsync`), vérification de l'enrôlement biométrique de l'utilisateur (`isEnrolledAsync`), puis déclenchement de l'invite système avec gestion du fallback code PIN lorsque la biométrie est indisponible.

Listing 7 – Preuve d'implémentation : src/screens/OrdonnancesScreen.tsx

```tsx
const checkBiometricAndAuthenticate = useCallback(async () => {
  if (!(await LocalAuthentication.hasHardwareAsync()) || !(await LocalAuthentication.isEnrolledAsync
      ())) {
    return handleFallbackAuth(); // Fallback securise PIN / mot de passe applicatif
  }
  const result = await LocalAuthentication.authenticateAsync({
    promptMessage: "Authentification requise pour acceder a vos ordonnances",
    cancelLabel: "Annuler",
    disableDeviceFallback: false,
  });
  if (result.success) {
    setIsAuthenticated(true);
    await loadOrdonnances(); // Appel API protege par Bearer token
  }
}, []);
```

5.7. Bonus : Proxy Sécurisé pour l'API Google Maps (+1 pt)

La clé d'API est retirée du binaire et déplacée côté serveur. Un endpoint mandataire (`GET /v1/maps/geocode`), accessible uniquement aux patients authentifiés, transmet les requêtes de géocodage avec un contrôle de débit (*rate limiting*) par utilisateur. Sur Google Cloud Console, la clé est restreinte par empreinte SHA-256

et Package Name / Bundle ID, et scoped à la seule API Geocoding avec des quotas stricts, empêchant toute réutilisation frauduleuse même en cas d'exfiltration.

Listing 8 – Preuve d'implémentation : server/server.js

```
app.get("/v1/maps/geocode", authenticateToken, async (req, res) => {
  const { address } = req.query;
  if (!address || typeof address !== "string" || address.length > 200) {
    return res.status(400).json({ error: "Parametre address invalide (max 200 car)." });
  }
  const url = `https://maps.googleapis.com/maps/api/geocode/json?address=${encodeURIComponent(
      address)}&key=${process.env.GOOGLE_MAPS_API_KEY}`;
  const response = await fetch(url);
  res.json(await response.json());
});
```

6. Analyses Critiques

6.1. Hermes Bytecode : Optimisation ou Protection ?

Hermes compile le JavaScript en bytecode (`.hbc`) dans l'objectif d'accélérer le temps de démarrage et de réduire l'empreinte mémoire, non de protéger le code propriétaire. Les chaînes de caractères demeurent dans une *String Pool* non chiffrée, et la commande `strings index.android.bundle | grep -E "AIza|api.santeplus"` les expose immédiatement. Des outils tels que `hermes-dec` ou `hbctool` permettent de désassembler la totalité des instructions et de reconstituer la logique applicative.

La sécurité ne peut pas reposer sur l'obscurité : tout secret doit demeurer côté serveur, et le code JavaScript sensible doit être soumis à une offuscation professionnelle (Jscrambler, ProGuard/R8) faisant partie intégrante du pipeline de build.

6.2. Détection Root / Jailbreak : Portée Réelle

Une bibliothèque telle que `jail-monkey` ne constitue pas une barrière suffisante en isolation : un attaquant peut la neutraliser en quelques lignes de script Frida, par exemple :

```
JailMonkey.isJailBroken.implementation = () => false;
```

ou de façon entièrement transparente via le module kernel Magisk DenyList / Shamiko.

Son utilité repose sur une logique de **défense en profondeur** (MASVS-RESILIENCE) : elle bloque la majorité des logiciels espions automatisés non ciblés. Couplée à une attestation distante : **Play Integrity API** sur Android et **App Attest** sur iOS, elle permet au serveur de refuser la délivrance de tokens à des environnements compromis, transformant une vérification client contournable en décision serveur non manipulable.

6.3. Mises à Jour OTA (EAS Update) : Risques et Durcissement

Les mises à jour OTA court-circuitent les audits de sécurité des App Stores et constituent le principal vecteur de *Supply Chain Attack* (M2) sur les applications Expo : une compromission des identifiants de pipeline CI/CD permet d'injecter un bundle JavaScript malveillant sur l'intégralité du parc en production sans intervention humaine.

Les contre-mesures indispensables sont au nombre de trois :

- **Signature de code (Code Signing) :** chaque bundle est signé par une clé privée RSA/ECDSA conservée hors ligne ; le binaire natif vérifie la signature cryptographique avant tout rechargement, rendant l'injection d'un bundle non signé impossible même en cas de compromission de l'infrastructure EAS.
- **Cloisonnement des canaux :** séparation stricte entre les environnements `development`, `staging` et `production` ; toute promotion d'un bundle entre canaux requiert une approbation explicite.
- **Gouvernance :** MFA obligatoire (FIDO2/passkey) et politique de moindre privilège (RBAC) sur l'organisation EAS ; rotation régulière des secrets de CI/CD avec traçabilité complète des déploiements.

7. Plan de Vérification

Pour garantir l'efficacité des correctifs et la conformité continue avec les exigences de la CNDP (Loi 09-08) et de l'OWASP MASVS, les tests suivants doivent être exécutés avant chaque mise en production :

#	Test	Procédure de test	Critère de succès attendu
V1	Extraction de secrets du bundle	`apktool d app.apk` puis commande `strings` sur le bundle JS	Aucun secret, token ou clé API Google Maps trouvé en clair
V2	Interception MITM	Proxy mitmproxy avec certificat CA utilisateur installé sur le terminal	Connexion TLS refusée catégoriquement (Certificate Pinning)
V3	Lecture du stockage local	Commande `adb shell run-as` et tentative d'accès SQLite	Données inaccessibles / chiffrées matériellement (Keystore)
V4	Élévation de privilèges	Hook Frida de `isAdmin()` et altération du token JWT	Accès refusé par le serveur avec code HTTP 403 Forbidden
V5	Deep link malveillant	Émission de `santeplus://rdv?id=../../admin` via adb/SMS	Lien rejeté immédiatement par la validation regex UUID v4
V6	Injection WebView (RCE)	Exécution de `postMessage` avec payload arbitraire / script	Message ignoré sans exécution (filtrage par liste blanche)
V7	Scan automatisé MobSF	Soumission de l'APK de release sur la plateforme MobSF	Aucun avertissement critique, score de sécurité $\geq 85/100$
V8	Biométrie ordonnances	Tentative d'accès à l'écran sans empreinte ou Face ID valide	Écran strictement verrouillé, invite fallback PIN exigée

8. Glossaire

Acronyme	Définition
ADB	Android Debug Bridge, outil de communication avec un terminal Android
APK	Android Package Kit, format d'archive d'une application Android
CA	Certificate Authority, autorité de certification
C2	Command and Control, serveur de pilotage d'un malware
CVSS	Common Vulnerability Scoring System, échelle standard de criticité
DAST	Dynamic Application Security Testing, test de sécurité dynamique
FIDO2	Fast Identity Online 2, standard d'authentification sans mot de passe
Frida	Framework d'instrumentation dynamique pour applications mobiles
JWT	JSON Web Token, jeton d'authentification signé (RFC 7519)
MASVS	Mobile Application Security Verification Standard (OWASP)
MITM	Man-in-the-Middle, attaque par interception de communications
MobSF	Mobile Security Framework, outil d'audit automatisé
OTA	Over-the-Air, mise à jour à distance sans passage par l'App Store
RBAC	Role-Based Access Control, contrôle d'accès par rôles
RCE	Remote Code Execution, exécution de code à distance
SAST	Static Application Security Testing, test de sécurité statique
SPKI	Subject Public Key Info, empreinte de clé publique pour le pinning
TEE	Trusted Execution Environment, zone d'exécution sécurisée du processeur
TLS	Transport Layer Security, protocole de chiffrement des communications

9. Conclusion et Perspectives

La refonte de **SantéPlus** vers `santeplus-secure` illustre un principe clé de la sécurité mobile : **le terminal client est un environnement non maîtrisé qu'il ne faut jamais considérer comme digne de confiance**. L'audit initial a montré qu'une application conçue sans approche *Security by Design* expose directement les données médicales des patients à des interceptions réseau, des extractions locales ou des exécutions de code à distance.

La mise en œuvre des mécanismes de sécurisation — chiffrement matériel des jetons via Keystore et Keychain (`expo-secure-store`), flux HTTPS avec jeton *Bearer*, validation des liens profonds et isolation des `WebView` — neutralise les 14 vulnérabilités identifiées. Couplée au déport des contrôles d'autorisation (RBAC/JWT) et des clés API vers un serveur proxy sécurisé, ainsi qu'à la biométrie obligatoire sur les ordonnances, cette architecture répond aux exigences de l'**article 23 de la Loi 09-08** (CNDP) et de l'**OWASP MASVS v2.0**.

Toutefois, la sécurité mobile demeure un processus continu. Pour pérenniser cette protection en production, cette base devra être complétée par des analyses statiques automatisées (SAST) dans la chaîne CI/CD, un mécanisme de *certificate pinning* avec rotation des clés et des tests d'intrusion réguliers avant chaque mise à jour majeure.

Références

[1] OWASP Foundation, *Mobile Top 10 (2024 Edition)*, https://owasp.org/www-project-mobile-top-10/.

[2] OWASP Foundation, *Mobile Application Security Verification Standard v2.0*, https://mas.owasp.org/MASVS/.

[3] CNDP Maroc, *Loi n°09-08 relative à la protection des personnes physiques*, Bulletin Officiel n°5714, https://www.cndp.ma/loi-09-08/.

[4] React Native Community, *Security Guidelines and Best Practices*, https://reactnative.dev/docs/security.

[5] MobSF Project, *Mobile Security Framework Documentation*, https://mobsf.github.io/docs/.